1 Prime atoms, prime bags, and prime multisets

Source: `PrimeTensor/Prime.lean`

What this file establishes

The file builds the multiplicative carrier the project starts from, in two stages. First an *ordered* factor chain, PrimeBag, whose terminator is the multiplicative pivot `one` and whose multiplication is concatenation. Concatenation is associative and unital but visibly not commutative: the chain $2 \cdot 3$ and the chain $3 \cdot 2$ are different terms. Second, the quotient PrimeMultiset of chains by equality of represented magnitude, on which multiplication *is* commutative — not up to a relation, but as an equality of elements.

So the file's arc is: pick a representation in which the operation is computable and order-sensitive, then quotient exactly enough that order stops being observable at the public boundary.

1.1 Prime atoms

Definition 1.1 (Prime, `PrimeTensor.Prime`)**.** Prime is the structure with fields

$$\text{value} : \mathbb{N}, \qquad \text{prime} : \text{Nat.Prime}(\text{value}).$$

```
structure Prime where
  value : ℕ
  prime : Nat.Prime value
```

An atom is a natural number bundled with a proof that it is prime, so primality travels with the number and never has to be re-established. Two atoms are equal exactly when their value fields are — the proof field is a proposition, hence proof-irrelevant.

Lemma 1.2 (`PrimeTensor.Prime.one_lt`)**.** *For every p : Prime, $1 < p$.value.*

Proof. This is `Nat.Prime.one_lt` applied to the stored field p.prime. It is tagged `@[simp]` so that the strict lower bound on an atom is available to the simplifier without being re-derived. □

1.2 Prime bags

Definition 1.3 (Prime bag, `PrimeTensor.PrimeBag`)**.** PrimeBag is the inductive type generated by

$$\mathsf{one} : \text{PrimeBag}, \qquad \mathsf{factor} : \text{Prime} \to \text{PrimeBag} \to \text{PrimeBag}.$$

```
inductive PrimeBag where
  | one : PrimeBag
  | factor : Prime → PrimeBag → PrimeBag
```

A bag is a finite list of atoms terminated by the pivot; the name records the intent, but at this stage the structure is a list and the order of factors is part of the term. As with Depth in `PrimeTensor/Depth.lean`, the terminator is `one`, the multiplicative unit, and not a zero.

Definition 1.4 (Evaluation, `PrimeTensor.PrimeBag.eval`)**.** eval : PrimeBag → ℕ is defined by structural recursion:

$$\mathrm{eval}(\mathsf{one}) = 1, \tag{1}$$

$$\mathrm{eval}(\mathsf{factor}\ p\ r) = p.\mathrm{value} \cdot \mathrm{eval}(r). \tag{2}$$

```
def eval : PrimeBag → ℕ
  | .one => 1
  | .factor p rest => p.value * rest.eval
```

Lemma 1.5 (`PrimeBag.eval_one`, `PrimeBag.eval_factor`)**.** *Equations* (1) *and* (2) *hold.*

Proof. Each is the defining equation of eval at a constructor, so both sides are definitionally equal and the proof is `rfl`. Both are tagged `@[simp]`: the statements are content-free, and the tag is what makes them usable by the simplifier, which does not unfold recursive definitions on its own. □

Lemma 1.6 (`PrimeTensor.PrimeBag.one_le_eval`)**.** *For every* b : PrimeBag, $1 \le \mathrm{eval}(b)$.

Proof. By structural induction on b.

For $b = \mathsf{one}$ the goal is $1 \le 1$, closed by reflexivity of $\le$.

For $b = \mathsf{factor}\ p\ r$, Lemma 1.2 gives $1 < p.\mathrm{value}$ and hence $1 \le p.\mathrm{value}$, while the induction hypothesis gives $1 \le \mathrm{eval}(r)$. Monotonicity of multiplication on ℕ in both arguments then yields

$$1 = 1 \cdot 1 \ \le\ p.\mathrm{value} \cdot \mathrm{eval}(r) = \mathrm{eval}(\mathsf{factor}\ p\ r). \qquad \square$$

So evaluation lands in $\mathbb{Z}_{\ge 1}$: no bag represents 0. This is the same decision one layer up from the absence of a zero depth in `PrimeTensor/Depth.lean` — the carrier is positive by construction, so the multiplicative structure below never meets an absorbing element.

1.3 Multiplication is concatenation

Definition 1.7 (Product of bags, `PrimeTensor.PrimeBag.mul`)**.** mul : PrimeBag → PrimeBag → PrimeBag is defined by structural recursion on its first argument:

$$\mathsf{one} \cdot b = b, \tag{3}$$

$$(\mathsf{factor}\ p\ r) \cdot b = \mathsf{factor}\ p\ (r \cdot b). \tag{4}$$

The instances `One PrimeBag` and `Mul PrimeBag` make 1 denote one and · denote mul.

```
def mul : PrimeBag → PrimeBag → PrimeBag
  | .one, b => b
  | .factor p rest, b => .factor p (mul rest b)

instance : One PrimeBag := ⟨.one⟩
instance : Mul PrimeBag := ⟨mul⟩
```

This is list concatenation: the factors of a are re-emitted in order and the chain is then continued by b.

Lemma 1.8 (Left unit, `PrimeTensor.PrimeBag.one_mul`)**.** $1 \cdot b = b$ *for every* b : PrimeBag.

Proof. This is (3), the defining equation at `one`. The recursion matches on the first argument, which is already a constructor here, so the two sides are definitionally equal and the proof is `rfl`. □

Lemma 1.9 (Right unit, `PrimeTensor.PrimeBag.mul_one`)**.** $b \cdot 1 = b$ *for every* $b :$ PrimeBag.

Proof. By structural induction on b; unlike Lemma 1.8 this is *not* definitional, because the recursion inspects the first argument and b is a variable.

For $b = \mathsf{one}$, (3) gives $\mathsf{one} \cdot 1 = 1 = \mathsf{one}$, again by reflexivity.

For $b = \mathsf{factor}\ p\ r$, (4) rewrites the goal to $\mathsf{factor}\ p\ (r \cdot 1) = \mathsf{factor}\ p\ r$, and the induction hypothesis $r \cdot 1 = r$ gives it after congruence under $\mathsf{factor}\ p\ (-)$. □

Lemma 1.10 (Associativity, `PrimeTensor.PrimeBag.mul_assoc`)**.** $(a \cdot b) \cdot c = a \cdot (b \cdot c)$ *for all* $a, b, c :$ PrimeBag.

Proof. By structural induction on a, with b and c fixed.

For $a = \mathsf{one}$ both sides reduce to $b \cdot c$ by (3), so the case is definitional.

For $a = \mathsf{factor}\ p\ r$, two applications of (4) on each side turn the goal into $\mathsf{factor}\ p\ ((r \cdot b) \cdot c) = \mathsf{factor}\ p\ (r \cdot (b \cdot c))$, which follows from the induction hypothesis by congruence under $\mathsf{factor}\ p\ (-)$. □

Lemma 1.11 (Evaluation is multiplicative, `PrimeTensor.PrimeBag.eval_mul`)**.** $\mathrm{eval}(a \cdot b) = \mathrm{eval}(a) \cdot \mathrm{eval}(b)$ *for all* $a, b :$ PrimeBag.

Proof. By structural induction on a.

For $a = \mathsf{one}$ the left side is $\mathrm{eval}(b)$ by (3) and the right side is $1 \cdot \mathrm{eval}(b)$ by (1); these agree by the left unit law of $\mathbb{N}$.

For $a = \mathsf{factor}\ p\ r$ the two sides are, after (4) and (2),

$$p.\mathrm{value} \cdot \mathrm{eval}(r \cdot b) \qquad \text{and} \qquad (p.\mathrm{value} \cdot \mathrm{eval}(r)) \cdot \mathrm{eval}(b).$$

Rewriting the first by the induction hypothesis makes them $p.\mathrm{value} \cdot (\mathrm{eval}(r) \cdot \mathrm{eval}(b))$ and $(p.\mathrm{value} \cdot \mathrm{eval}(r)) \cdot \mathrm{eval}(b)$, equal by associativity in $\mathbb{N}$. □

Remark 1.12. Lemmas 1.8, 1.9 and 1.10 are exactly the monoid laws, and Lemma 1.11 says eval is a monoid homomorphism to $(\mathbb{N}, \cdot, 1)$. The file proves the laws but does not register a `Monoid` instance; only `One` and `Mul` are declared. What is deliberately absent is commutativity, which is false for PrimeBag: taking $p \neq q$,

$$\mathsf{factor}\ p\ (\mathsf{factor}\ q\ \mathsf{one}) \ \neq\ \mathsf{factor}\ q\ (\mathsf{factor}\ p\ \mathsf{one}),$$

since `factor` is injective in its first argument. (This non-equality is an observation for the reader; the file states neither it nor any commutativity law for PrimeBag.)

1.4 Identifying factor order

Definition 1.13 (Same, `PrimeTensor.PrimeBag.Same`)**.** For $a, b :$ PrimeBag,

$$\mathrm{Same}(a, b) \equiv (\mathrm{eval}(a) = \mathrm{eval}(b)).$$

Same is thus, by definition, the kernel of eval: two bags are identified exactly when they denote the same natural number.

Lemma 1.14 (`same_refl`, `same_symm`, `same_trans`)**.** Same *is reflexive, symmetric and transitive.*

Proof. Unfolded, each statement is the corresponding property of equality on $\mathbb{N}$, so the three proofs are `rfl`, Eq.symm and Eq.trans respectively. They carry `@[refl]`, `@[symm]` and `@[trans]` so that the relational tactics can use them directly. □

Definition 1.15 (The setoid, `PrimeTensor.PrimeBag.setoid`)**.** setoid : Setoid(PrimeBag) is Same together with the equivalence proof assembled from Lemma 1.14.

Lemma 1.16 (Concatenation commutes up to Same, `PrimeTensor.PrimeBag.mul_comm_same`)**.** $\mathrm{Same}(a \cdot b,\ b \cdot a)$ *for all* a, b : PrimeBag*.*

Proof. Unfolding Definition 1.13, the goal is $\mathrm{eval}(a \cdot b) = \mathrm{eval}(b \cdot a)$. Two applications of Lemma 1.11 reduce it to $\mathrm{eval}(a) \cdot \mathrm{eval}(b) = \mathrm{eval}(b) \cdot \mathrm{eval}(a)$, which is commutativity of multiplication on $\mathbb{N}$. □

Lemma 1.17 (Same is a congruence, `PrimeTensor.PrimeBag.mul_same`)**.** *If* $\mathrm{Same}(a_1, a_2)$ *and* $\mathrm{Same}(b_1, b_2)$ *then* $\mathrm{Same}(a_1 \cdot b_1,\ a_2 \cdot b_2)$*.*

Proof. By Lemma 1.11 the goal unfolds to $\mathrm{eval}(a_1) \cdot \mathrm{eval}(b_1) = \mathrm{eval}(a_2) \cdot \mathrm{eval}(b_2)$, and rewriting with the two hypotheses closes it. □

Remark 1.18. Both proofs are short for the same structural reason: Same was defined as a property of eval alone, and eval was already known to be multiplicative. Every fact about the relation is therefore imported from $\mathbb{N}$ rather than proved about factor chains. Had Same instead been defined as "is a permutation of", both lemmas would have needed genuine combinatorial work.

1.5 Prime multisets

Definition 1.19 (`PrimeTensor.PrimeMultiset`)**.**

$$\mathrm{PrimeMultiset} \equiv \mathrm{Quotient}(\mathrm{PrimeBag.setoid}),$$

declared as an `abbrev`, hence reducible. Write $\mathrm{ofBag}(b)$ for the class of b, and $\mathrm{one} \equiv \mathrm{ofBag}(1)$.

Definition 1.20 (Induced operations)**.** The declarations `PrimeMultiset.eval` and `PrimeMultiset.mul`:

- eval : PrimeMultiset $\to \mathbb{N}$ is the bag-level evaluation of Definition 1.4, lifted through the quotient;
- mul is $(x, y) \mapsto \mathrm{ofBag}(x \cdot y)$ lifted through the quotient in both arguments.

The instances `One PrimeMultiset` and `Mul PrimeMultiset` give these the usual notation.

Each lift carries a well-definedness obligation, and the two are discharged differently:

- For eval the obligation is that $\mathrm{Same}(a, b)$ implies $\mathrm{eval}(a) = \mathrm{eval}(b)$. By Definition 1.13 that implication is the identity: the hypothesis *is* the conclusion. This is the payoff of quotienting by the kernel of the very map one wants to lift.

- For mul the obligation is that Same is respected in both arguments, which is Lemma 1.17, followed by Quotient.sound to turn the resulting Same back into an equality of classes.

Lemma 1.21 (`eval_ofBag`, `eval_one`, `eval_mul`)**.** *For* $b :$ PrimeBag *and* $a, b' :$ PrimeMultiset*,*

$$\mathrm{eval}(\mathrm{ofBag}(b)) = \mathrm{eval}(b), \qquad \mathrm{eval}(1) = 1, \qquad \mathrm{eval}(a \cdot b') = \mathrm{eval}(a) \cdot \mathrm{eval}(b').$$

Proof. The first holds by rfl: a lift applied to a representative reduces definitionally to the underlying function. The second is the first at $b =$ one together with (1). The third is proved by Quotient.inductionOn$_2$ — it suffices to check the identity on representatives x, y, where it becomes $\mathrm{eval}(x \cdot y) = \mathrm{eval}(x) \cdot \mathrm{eval}(y)$, which is Lemma 1.11. All three are tagged `@[simp]`. □

Proposition 1.22 (Commutativity, `PrimeTensor.PrimeMultiset.mul_comm`)**.** $a \cdot b = b \cdot a$ *for all* $a, b :$ PrimeMultiset *— an equality of elements, not merely of represented values.*

Proof. By Quotient.inductionOn$_2$ it suffices to prove it for classes of representatives x, y, where the goal is $\mathrm{ofBag}(x \cdot y) = \mathrm{ofBag}(y \cdot x)$. By Quotient.sound this follows from $\mathrm{Same}(x \cdot y, y \cdot x)$, which is Lemma 1.16. □

This is the point of the construction. Concatenation of chains is not commutative (Remark 1.12); the quotient makes the induced operation commutative on the nose, so downstream files may rewrite with mul_comm as an ordinary equation and never carry a setoid or a "modulo reordering" side condition.

Remark 1.23 (What the quotient is, and what unique factorization is for)*.* It is worth being exact about the roles of the two arithmetic facts here, because the file's header mentions unique factorization while its proofs do not use it.

Since Same is the kernel of eval (Definition 1.13), the induced map

$$\mathrm{eval} : \mathrm{PrimeMultiset} \longrightarrow \mathbb{N}$$

is injective. By Lemma 1.6 its image lies in $\mathbb{Z}_{\geq 1}$, and since every $n \geq 1$ is a product of primes the image is all of $\mathbb{Z}_{\geq 1}$. So PrimeMultiset is isomorphic, as a commutative monoid, to $(\mathbb{Z}_{\geq 1}, \cdot, 1)$.

That reorderings are identified by the quotient needs only commutativity of $\mathbb{N}$, which is what Lemma 1.16 uses. Unique factorization is what gives the converse: that *only* reorderings are identified, so that a class really is the multiset of prime factors and not a coarser object. That converse justifies the name PrimeMultiset, but it is not proved in this file and no result here depends on it.

1.6 Concrete atoms and a worked value

Definition 1.24 (`primeTwo`, `primeThree`, `primeFive`, `twelveBag`, `twelve`)**.** primeTwo, primeThree and primeFive are the atoms 2, 3 and 5, each paired with a primality proof discharged by `norm_num`. Further,

$$\mathrm{twelveBag} \equiv \mathsf{factor}\ 2\ (\mathsf{factor}\ 2\ (\mathsf{factor}\ 3\ \mathsf{one})), \qquad \mathrm{twelve} \equiv \mathrm{ofBag}(\mathrm{twelveBag}).$$

Unfolding Definition 1.4 three times,

$$\mathrm{eval}(\mathrm{twelveBag}) = 2 \cdot (2 \cdot (3 \cdot 1)) = 12,$$

and primeFive is declared but not used elsewhere in the file. Note that twelveBag fixes an order for the factors of 12 while twelve does not: any of the three arrangements of $2, 2, 3$ gives a bag with the same image under ofBag, by Proposition 1.22.

Summary of the correspondence

Lean declaration	Kind	Here
`PrimeTensor.Prime`	structure	Def. 1.1
`Prime.one_lt`	simp thm	Lem. 1.2
`PrimeTensor.PrimeBag`	inductive	Def. 1.3
`PrimeBag.eval`	definition	Def. 1.4
`PrimeBag.eval_one`	simp thm	Lem. 1.5
`PrimeBag.eval_factor`	simp thm	Lem. 1.5
`PrimeBag.one_le_eval`	theorem	Lem. 1.6
`PrimeBag.mul`	definition	Def. 1.7
`One PrimeBag`, `Mul PrimeBag`	instances	Def. 1.7
`PrimeBag.one_mul`	simp thm	Lem. 1.8
`PrimeBag.mul_one`	simp thm	Lem. 1.9
`PrimeBag.mul_assoc`	theorem	Lem. 1.10
`PrimeBag.eval_mul`	simp thm	Lem. 1.11
`PrimeBag.Same`	definition	Def. 1.13
`PrimeBag.same_refl`	refl thm	Lem. 1.14
`PrimeBag.same_symm`	symm thm	Lem. 1.14
`PrimeBag.same_trans`	trans thm	Lem. 1.14
`PrimeBag.setoid`	definition	Def. 1.15
`PrimeBag.mul_comm_same`	theorem	Lem. 1.16
`PrimeBag.mul_same`	theorem	Lem. 1.17
`PrimeTensor.PrimeMultiset`	abbrev	Def. 1.19
`PrimeMultiset.ofBag`	definition	Def. 1.19
`PrimeMultiset.one`	definition	Def. 1.19
`PrimeMultiset.eval`	definition	Def. 1.20
`PrimeMultiset.mul`	definition	Def. 1.20
`One PrimeMultiset`, `Mul PrimeMultiset`	instances	Def. 1.20
`PrimeMultiset.eval_ofBag`	simp thm	Lem. 1.21
`PrimeMultiset.eval_one`	simp thm	Lem. 1.21
`PrimeMultiset.eval_mul`	simp thm	Lem. 1.21
`PrimeMultiset.mul_comm`	theorem	Prop. 1.22
`primeTwo`, `primeThree`, `primeFive`	definitions	Def. 1.24
`twelveBag`, `twelve`	definitions	Def. 1.24

Every declaration of `PrimeTensor/Prime.lean` appears above. The file contains no `sorry`, no axiom, and no unproved named proposition. The one statement in this section that is not backed by a declaration in the file is Remark 1.23's identification of PrimeMultiset with $(\mathbb{Z}_{\geq 1}, \cdot, 1)$, which is stated for the reader.